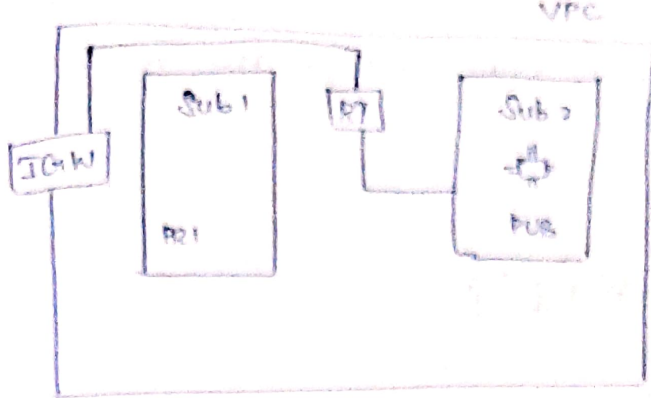




6/10/2026

Day 8

In ec2,

Documentation → VPC → aws security group.

→ ingress rule - "allow-ssh" ref. name it has port number for port 22.

cidr-ipv4 - "0.0.0.0/0"

"allow-http" port 80

→ vpc-security-group-id = [aws-security-group-allow-lls.id]

VPC create partition, attach EC2 and partition, then security group add partition.

terraform variables:

- use: reusability

can use variable

"variable.tf"

ref.name

"ec2.tf"

variable "ami-reference" {

ami = var.ami-reference

type = string

default = "ami-od2..."

}

ubuntu AMI id

- direct
- environment
- terraform.tfvars
- variable.tf

variable type methods

→ least priority

variables.tf ⇒ default value set pandra mai.

variables.tf :

```
variable "ami-ref" {
```

```
  type = string
```

```
  default = "ami-od..." # ubuntu ami
```

```
}
```

terraform.tfvars:

```
ami-ref = "ami-od24..." # amazon linux
```

"variables.tf ku least priority becoz it is default value mai. so when you change it nee default value than kuptu edit pannu ur wish ku matha mudiyum. Athaan tfvars la ref name pottu edit pandrom."

• General ah instance oda public id terraform.tfstate la irukum. All details athula irukum, but evu evu rathiyum nuaiya details la find pandrathu tough. So, we use outputs.tf file

→ created outputs.tf.

```
→ output "testec2-public-ip" {
```

```
  value = ref-name.public-ip
```

```
}
```

→ save

→ terraform apply

→ Output value as,

testec2 - public-ip = " ip "

ssh -i " private key path " username

↓, in format

• pem

eg: Ubuntu @ 54.242.9.126

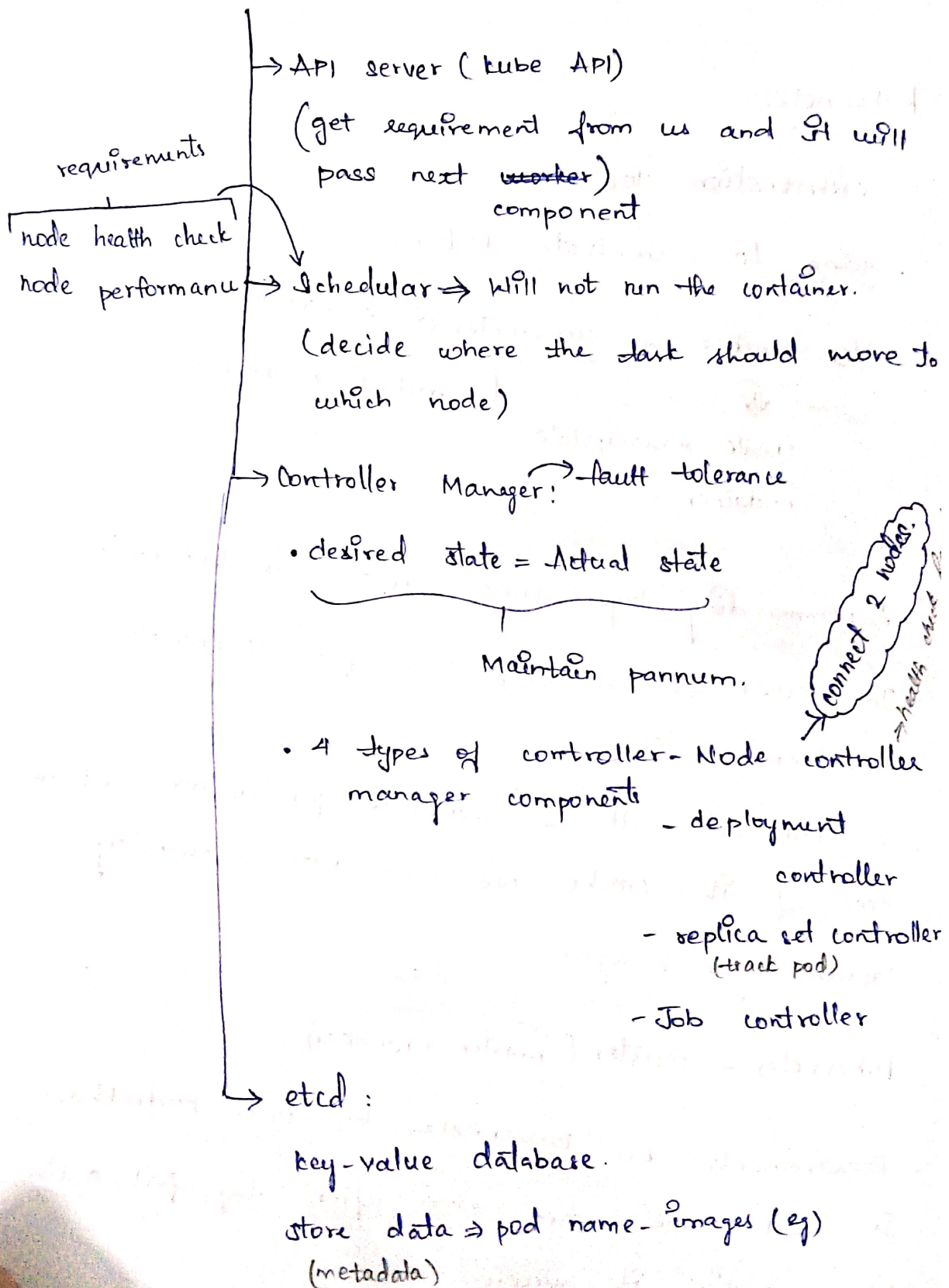
Kubernetes:

- orchestration tool
- going to orchestrate docker.
- d/b docker & kubernetes (X) (X) Interview
 ↓
 create & manipulate container
 ↳ orchestrate docker
- A commanding layer above docker is kubernetes.
- In Docker, we are responsible for all operation in container.
- In kubernetes, it is responsible for docker container even if it crashes we need not manually launch again & again.
- kubernetes - cluster (nodes)
 (master + worker)
- Requirements on kubernetes like once kubernetes, it will take care even after dry, fail, again repeat until it success.
- mini kube → no master cluster.
 ↳ can run commands.

In EKS, master → AWS will take care the responsibility
worker → we will create and we'll take care

* Master node or Control Plane:

- 4 components



pod - manage containers.

kubernetes will launch pod not containers.

- Pod - container run aagura smallest deployable unit.
 - one or more containers - ah kubernetes manage panha use panha wrapper.
 - IP address Irukum for pod.
 - same network and volume for a pod, it may have many containers.

- 1) container run pannum
- 2) container crash aana restart pannum
- 3) Multiple containers iruntha manage/scale pannum
- 4) Traffic - ah containers lu distribute pannum.
- 5) New version deploy panna help pannum.

* Worker components:

- i) pod
- ii) container runtime (container d)
- iii) kubelet [communication b/w worker & master node]
- iv) kube proxy. [used for port forwarding]

7/10/2026

Day-9

Resource Object

Minikube:

- minikube version
- minikube start